

TP2: Outils Gcc et make

1 Getting started

L'objectif de ce laboratoire est de vous familiariser avec le langage C et sa chaîne de développement de développement basée sur le compilateur **gcc**, et de vous apprendre à créer et à utiliser les fichiers **makefile** lorsque vous développez des programmes en C.

2 Un programme C composé d'un seul fichier de compilation

1. Créez un nouveau répertoire pour ce laboratoire dans votre espace personnel et copiez-y le contenu du répertoire `/share/l3info/CUnix/tp2` en utilisant la commande **cp -r**
2. A l'intérieur du répertoire copié, compilez le programme **hms.c** en utilisant la commande :
gcc -Wall filename.c
Que signifie l'option **-Wall** ?
3. Ouvrez le fichier source avec un éditeur et corrigez les erreurs.
4. Recompilez le programme pour vérifier que le programme est maintenant correct.
5. Utilisez la commande **ls -ail** pour identifier le fichier produit à la suite du processus de compilation. Quel est ce type de fichier ?
6. Pour spécifier un nom d'exécutable particulier pour le fichier produit, nous pouvons utiliser l'option **-o** de la commande **gcc** :
gcc -o progname -Wall filename.c
Nommez le fichier exécutable de **hms.c** comme **hms** (les fichiers exécutables de Linux n'ont pas l'extension `.exe` comme sous DOS / Window).
7. Exécutez le programme en utilisant `./` et vérifiez que sa fonctionnalité est correcte.

3 Un programme C composé de plusieurs fichiers de compilation

En règle générale, un fichier de compilation est constitué d'un fichier d'extension `.c` contenant le code source et d'un fichier d'en-tête d'extension `.h`, qui permet d'exporter les définitions des identifiants du module (données, fonctions, etc.).

Lorsque nous compilons un fichier de compilation, nous utilisons l'option **-c** du compilateur. Cela produit un fichier objet avec l'extension `.o` qui n'est pas directement exécutable.

1. Entrez dans le répertoire **tp_liste**, que vous avez déjà copié depuis l'exercice précédent. Ce répertoire contient le répertoire **src** où sont stockés les fichiers sources et le répertoire **include** où sont stockés les fichiers d'en-tête. Les fichiers sources contiennent le code source (partiel) de l'implémentation des listes chaînées en C.
2. Compilez les fichiers **src/test_list.c** en utilisant la commande :
gcc -Wall -c src/test_list.c
3. Cela produit une erreur de compilation, d'où vient cette erreur ? Comment corriger cette erreur ?
 - **Index 1** : nous avons un fichier d'en-tête **list.h** qui définit les variables et les identifiants exportés par le fichier **list.c** stocké dans le répertoire **include**, et la directive **#include "file.h"**.
 - **Index 2**: il est parfois nécessaire d'indiquer au compilateur où trouver les fichiers d'en-tête, s'ils ne se trouvent pas dans le même répertoire. Pour ce faire, nous pouvons utiliser l'option **-I** de la **gcc** :
gcc -I./include
qui indique que vous devez rechercher les fichiers d'en-tête **.h** dans le répertoire **./include**.
4. Une fois le problème résolu, compilez (avec l'option **-c**) le fichier **src/list.c**. Quel est le résultat de cette commande ? Ce fichier est-il un fichier exécutable ?
5. Nous devons connecter les deux fichiers objets pour obtenir le programme exécutable final. Pour ce faire, utilisez la commande :
gcc -o test_list test_list.o list.o
6. Exécutez le programme et observez les résultats sur la sortie standard.
7. Le message d'erreur de segmentation indique que le programme a essayé d'accéder à une adresse mémoire illégale, soit à cause d'un débordement de tableau, soit à cause de l'utilisation d'un pointeur incorrect ou non initialisé.
8. Recompilez le fichier **test_list.c** en utilisant l'option **-Wall**. Que recevez-vous comme résultat ?
9. En vous basant sur les avertissements fournis par le compilateur, corrigez les erreurs dans le module **test_list.c** et vérifiez ensuite le bon fonctionnement du programme.

4 L'outil make

Chaque fois que vous modifiez un des fichiers de compilation (**.c** ou **.h**), vous devez recompiler tous les fichiers de compilation dépendants, et refaire la liaison entre les fichiers objets pour obtenir un fichier exécutable à jour. Cette tâche est longue à réaliser manuellement, surtout lorsque le programme comprend de nombreux fichiers de compilation (par exemple, le noyau du système d'exploitation Linux qui contient plusieurs centaines de fichiers de compilation C.).

Le programme **make** permet d'automatiser cette tâche en recompilant à chaque fois uniquement les fichiers qui doivent être compilés.

1. Complétez le **makefile** du répertoire **tp2** pour permettre la compilation et la liaison correctes des fichiers de compilation **test_list.c** et **list.c** selon les instructions suivantes :
 - Les fichiers **.o** produits lors de la compilation sont stockés dans le répertoire **./obj**.
 - Le programme exécutable s'appelle **test_list** et il est stocké dans le répertoire **./bin**.

— Que les fichiers objets et l'exécutable sont à jour avec leur code source **.c** dépendant et leur code source **.c**. **.h** fichiers d'en-tête dépendants.

2. Vérifiez le bon fonctionnement de votre **makefile** en modifiant l'un des fichiers, puis en re-exécutant le **makefile** à l'aide de la commande **make**.

3. Dans le fichier **makefile**, complétez l'entrée "clean" afin de supprimer tous les fichiers objets **.o** ainsi que l'exécutable produit.

4. Dans le fichier **makefile**, complétez l'entrée "listing" afin de produire un fichier pdf à partir des sources des fichiers **.c** et **.h**, en utilisant la séquence de commandes :

```
a2ps -o listing.ps include/list.h src/list.c src/test_list.c
ps2pdf listing.ps
rm listing.ps
```

5. Vérifiez que cette règle fonctionne correctement en l'appelant avec la commande :
make listing

6. Si vous souhaitez développer un programme en utilisant un "débugueur", vous devez le compiler (et tous ses modules) avec l'option **-g** comme indiqué ci-dessous :

```
gcc -Wall -g -o test_list.o test_list.c
```

Pour éviter de devoir modifier toutes les règles du fichier makefile à chaque fois, nous pouvons définir des variables plus génériques. Par exemple, en ajoutant la ligne suivante au début du **makefile**, cela permet de définir une variable **CFLAGS**.

```
CFLAGS = -Wall -g
```

Ensuite, on peut faire référence à cette variable en écrivant **\$(CFLAGS)** les règles du **makefile**, comme dans l'exemple ci-dessous :

```
gcc $(CFLAGS) -c module1.c
```

Modifiez le **makefile** pour que les appels à gcc soient effectués avec l'option **-g**.